Studio Nx

P1-05 로그인 API — 함수 설명 합본 (26~34)

2026-10-07

목차

1. ai-notes/backend(fastAPI_Python)/P1-05 - 로그인 API/26_security_create_access_token.md
2. ai-notes/backend(fastAPI_Python)/P1-05 - 로그인 API/27_security_create_refresh_token.md
3. ai-notes/backend(fastAPI_Python)/P1-05 - 로그인 API/28_security_decode_refresh_token.md
4. ai-notes/backend(fastAPI_Python)/P1-05 - 로그인 API/29_auth_service_authenticate.md
5. ai-notes/backend(fastAPI_Python)/P1-05 - 로그인 API/30_auth_service_get_active_user.md
6. ai-notes/backend(fastAPI_Python)/P1-05 - 로그인 API/31_auth_route_refresh_cookie.md
7. ai-notes/backend(fastAPI_Python)/P1-05 - 로그인 API/32_auth_route_login.md
8. ai-notes/backend(fastAPI_Python)/P1-05 - 로그인 API/33_auth_route_refresh.md
9. ai-notes/backend(fastAPI_Python)/P1-05 - 로그인 API/34_auth_route_logout.md

Backend(FastAPI/Python) 26 — `create_access_token()` · `_create_token()`

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-05 로그인 — Access Token 발급 · PRD 01 FR-103 · FR-105
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	25 입력 스키마 (P1-04 끝) · 27 `create_refresh_token()`

1. 하는 일

회원 번호로 **30분짜리 Access Token(JWT)** 을 만든다. 05 `decode_access_token()` 이 읽는 토큰을 만드는 쪽이다.

`create_access_token(7)` → `"eyJhbGciOiJIUzI1NiIs….eyJzdWIiOiI3Iiwi….kQ3x…"`

토큰 안의 칸	값	뜻
`sub`	`"7"`	토큰 주인 (회원 번호, 글자)
`type`	`"access"`	종류 — 리프레시 토큰(27)과 구분
`iat`	`1791339232`	만든 시각 (1970년부터 센 초)
`exp`	`1791341032`	만료 시각 = `iat` + 1800초 (30분)

함수	역할
`create_access_token()`	바깥에서 부르는 이름. 수명 30분을 정함
`_create_token()`	실제로 만드는 곳. 27 `create_refresh_token()` 도 같은 함수를 씀 (05 의 `_decode_token` 과 짝)

2. 언제 쓰나

쓰는 곳	화면	하는 일
P1-05 `POST /auth/login` (32)	home.html 로그인 모달	이메일 · 비밀번호가 맞으면 응답 `{"access_token": …}` 으로 줌
P1-05 `POST /auth/refresh` (33)	모든 페이지 (`api.js` 01)	리프레시 쿠키가 유효하면 새 Access Token 을 줌

받은 토큰은 브라우저 메모리에 두고(`setAccessToken`), 모든 API 요청의 `Authorization: Bearer …` 에 붙는다 (JS 02). 30분이 지나면 서버(05)가 401 을 주고, 브라우저가 33 으로 새 토큰을 받는다.

3. 코드

import (파일 맨 위)

```
import re
import string
from datetime import UTC, datetime, timedelta

import bcrypt
import jwt
```

```
from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
```

`security.py` 맨 위의 import 이다. **`from datetime import UTC, datetime, timedelta` 를 이번에 추가했다.** 이 함수들은 그것과 `jwt` · `get_settings` 를 쓴다.

import	쓰는 곳
`UTC` · `datetime` · `timedelta`	지금 시각(UTC), 30분이라는 기간
`jwt`	`jwt.encode` — 토큰 만들기 (05 의 `jwt.decode` 와 반대)
`get_settings`	`.env` 의 `JWT_SECRET` · `jwt_access_minutes`(30)

함수

```
JWT_ALGORITHM = "HS256"


def create_access_token(user_id: int) -> str:
    minutes = get_settings().jwt_access_minutes
    return _create_token(user_id, "access", timedelta(minutes=minutes))


def _create_token(user_id: int, token_type: str, lifetime: timedelta) -> str:
    now = datetime.now(UTC)
    payload = {
        "sub": str(user_id),
        "type": token_type,
        "iat": now,
        "exp": now + lifetime,
    }
    return jwt.encode(payload, get_settings().jwt_secret, algorithm=JWT_ALGORITHM)
```

문법 풀이

import 문법	뜻
`from datetime import UTC, datetime, timedelta`	파이썬 기본 `datetime` 모듈에서 세 이름을 가져옴

식	뜻
`timedelta(minutes=30)`	30분이라는 **기간**
`datetime.now(UTC)`	지금 시각 (UTC, 시간대 표시 있음)
`now + lifetime`	30분 뒤 시각
`jwt.encode(payload, 비밀값, algorithm="HS256")`	머리 · 내용 · 서명 세 부분을 만들어 `.` 으로 이은 글자. `datetime` 값은 PyJWT 가 초 단위 숫자로 바꿔 넣음
`str(user_id)`	`7` → `"7"`. JWT 규격에서 `sub` 는 글자
`_create_token` 의 `_`	이 파일 안에서만 쓰는 함수 (05 의 `_decode_token` 과 같은 관례)

4. 흐름

1. 설정에서 수명(30분)을 읽는다.

2. 지금 시각(UTC)을 잰다.
3. `sub` · `type` · `iat` · `exp` 를 담는다.
4. 서버 비밀값으로 서명해 토큰 글자를 돌려준다.

5. 왜 이렇게

선택	이유
30분	토큰은 가진 사람을 인정하는 방식(Bearer)이라, 훔쳐 가도 오래 쓰지 못하게 짧게 둔다. 대신 리프레시(27 · 33)로 사용자는 계속 로그인된 것처럼 쓴다
수명은 `.env` 설정에서	코드를 고치지 않고 바꿀 수 있다 (`jwt_access_minutes`)
`type` 을 넣음	05 가 `type` 을 검사해, 3일짜리 리프레시 토큰을 Access 자리에 쓰지 못하게 막는다
`iat`(만든 시각) 추가 — 템플릿에 없음	같은 회원이 같은 초에 두 번 받지 않는 한 토큰이 매번 달라진다. 나중에 "이 시각 이전 토큰은 무효" 같은 처리를 할 수 있다. 05 는 이 칸을 요구하지 않으므로 영향 없음
만들기를 `_create_token` 으로 분리	27 `create_refresh_token` 은 수명 · 종류만 다르다. 한 줄로 끝난다
알고리즘은 05 와 같은 `JWT_ALGORITHM`	만드는 쪽과 읽는 쪽이 한 상수를 쓴다

토큰 내용은 암호화되지 않는다 (05 §2). 누구나 읽을 수 있으므로 회원 번호 외에 이메일 · 권한 같은 것은 넣지 않는다.

6. 시험 (`tests/test_security.py` 에 5개 추가 · 모두 14개)

시험	확인한 것
만들고 읽기	`decode_access_token(create_access_token(7)) == 7` — 05 와 짝이 맞음
내용	`sub` = `"7"`, `type` = `"access"`, `exp - iat` = 1800초
머리	`alg` = `HS256`
시각	`iat` 가 지금과 2초 안, `exp` = `iat` + 1800
회원마다 다름	7번 · 8번 토큰이 다름

05 의 시험에서 쓰던 `jwt.encode` 직접 만들기(`make_token`)는 그대로 둔다. 만료 · 위조 같은 이상한 토큰은 이 함수로 만들 수 없기 때문이다.

전체 시험 191개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 27 — `create_refresh_token()`

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-05 로그인 — Refresh Token 발급 · PRD 01 FR-105 · FR-109
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	26 `create_access_token()` · 28 `decode_refresh_token()`

1. 하는 일

회원 번호로 **3일짜리 Refresh Token(JWT)** 을 만든다. 26 과 같은 모양이고 종류와 수명만 다르다.

	Access Token (26)	Refresh Token (이 함수)
`type`	`"access"`	`"refresh"`
수명	30분	**3일** (`jwt_refresh_days`)
브라우저 어디에	JS 메모리 (`api.js` 변수)	**HttpOnly 쿠키** `snx_refresh` — JS 가 읽을 수 없음
서버로 가는 곳	모든 API 의 `Authorization: Bearer …`	`/api/v1/auth/…` 요청에만 쿠키로 자동으로 실림
쓰임	API 요청	새 Access Token 받기 (33 `/auth/refresh`)

2. 언제 쓰나

쓰는 곳	하는 일
P1-05 `POST /auth/login` (32)	로그인 성공 → 이 토큰을 쿠키에 담아 응답 (31 `set_refresh_cookie`)

두 토큰이 함께 움직이는 모습

순서	일어나는 일
①	로그인 → Access(30분)는 응답 본문, Refresh(3일)는 쿠키
②	30분 동안 API 요청은 Access 로
③	Access 만료 → 서버 401 → 브라우저가 `POST /auth/refresh` (쿠키가 자동으로 실림)
④	서버가 Refresh 를 검사(28) → 새 Access(26)
⑤	3일이 지나면 Refresh 도 만료 → 다시 로그인

자동 로그인 체크와의 관계: 토큰의 수명은 **늘 3일**이다. 체크 여부는 **쿠키**의 수명만 바꾼다 (31).

로그인 모달의 "Keep me signed in"	쿠키	결과
체크	3일 쿠키	브라우저를 닫았다 열어도 3일 동안 로그인 유지
체크 안 함	세션 쿠키 (수명 없음)	브라우저를 닫으면 쿠키가 사라짐 → 다시 로그인

3. 코드

import (파일 맨 위)

```
import re
import string
from datetime import UTC, datetime, timedelta

import bcrypt
import jwt

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
```

`security.py` 맨 위의 import 이다 (26 과 같음, 이번에 추가한 것 없음). 이 함수는 **`timedelta`** · **`get_settings`** 를 쓰고, 만드는 일은 26 의 `_create_token()` 에 맡긴다.

함수

```
def create_refresh_token(user_id: int) -> str:
    days = get_settings().jwt_refresh_days
    return _create_token(user_id, "refresh", timedelta(days=days))
```

문법 풀이

import 문법	뜻
`from datetime import timedelta`	기간을 나타내는 타입 (26 에서 이미 가져옴)

식	뜻
`get_settings().jwt_refresh_days`	`.env` 설정. 기본 3 (`core/config.py`)
`timedelta(days=3)`	3일 = 259,200초
`_create_token(user_id, "refresh", …)`	26 과 같은 함수에 종류 · 수명만 다르게 넘김

4. 흐름

1. 설정에서 수명(3일)을 읽는다.
2. 26 의 `_create_token()` 으로 `type: "refresh"` 토큰을 만든다.

5. 왜 이렇게

선택	이유
Access 와 나눔	자주 오가는 Access 는 짧게(30분), 오래 가는 Refresh 는 JS 가 못 읽는 쿠키에 숨긴다. 스크립트 공격(XSS)이 Access 를 훔쳐도 30분 뒤면 못 쓴다
`type: "refresh"`	05 가 Access 자리에서 이 토큰을 거절하고, 28 은 반대로 Access 를 거절한다. 두 토큰을 서로 바꿔 쓸 수 없다
토큰 수명은 체크와 상관없이 3일	쿠키가 브라우저에서 사라지면 토큰도 쓸 일이 없다. 토큰을 두 가지로 만들 필요가 없다

선택	이유
같은 비밀값 · 알고리즘	26 과 같은 `_create_token` 을 써서 서명 방식이 하나로 유지된다

6. 시험 (`tests/test_security.py` 에 3개 추가 · 모두 17개)

시험	확인한 것
내용	`sub` = `"7"`, `type` = `"refresh"`, `exp - iat` = 259,200초 (3일)
Access 자리에 쓰기	05 `decode_access_token` 이 401 — 3일짜리 토큰으로 API 를 부를 수 없음
두 토큰은 다름	같은 회원의 Access 와 Refresh 가 다름

전체 시험 194개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 28 — `decode_refresh_token()`

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-05 로그인 — Refresh Token 검사 · PRD 01 FR-105
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	27 `create_refresh_token()` · 29 `authenticate()`

1. 하는 일

리프레시 쿠키의 토큰이 **진짜이고 아직 유효한 Refresh Token** 인지 확인하고 회원 번호를 돌려준다. 실패하면 401.

`decode_refresh_token("eyJ…")` → `7`

05 `decode_access_token()` 과 같은 검사이고, `type` 만 `"refresh"` 여야 한다.

함수	받아들이는 토큰
05 `decode_access_token()`	`type: "access"` 만
28 `decode_refresh_token()`	`type: "refresh"` 만

2. 언제 쓰나

쓰는 곳	화면	하는 일
P1-05 `POST /auth/refresh` (33)	모든 페이지 — `api.js` 의 `refreshAccessToken()` (JS 01)	쿠키 `snx_refresh` 의 토큰을 이 함수로 검사 → 회원 확인 → 새 Access Token

쿠키의 토큰	결과	브라우저
정상 (3일 안)	`7` → 새 Access Token	로그인 유지
만료 · 위조 · 종류가 `access` · 엉터리	401 "Please sign in again."	`refreshAccessToken()` 이 `false` → 로그인 화면으로

3. 코드

import (파일 맨 위)

```
import re
import string
from datetime import UTC, datetime, timedelta

import bcrypt
import jwt

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
```

`security.py` 맨 위의 import 이다 (이번에 추가한 것 없음). **이 함수는 import 한 것을 직접 쓰지 않는다** — 검사는 같은 파일의 05 `_decode_token()` 이 하고, 그 함수가 `jwt` · `get_settings` · `AppError` 를 쓴다.

함수

```
def decode_refresh_token(token: str) -> int:
    return _decode_token(token, "refresh")
```

05 에서 검사 부분을 `_decode_token(token, expected_type)` 으로 나눠 둔 덕분에 한 줄이다. 05 문서 §5 에 "P1-05 `decode_refresh_token()` 은 `return _decode_token(token, "refresh")` 한 줄이 된다" 고 적어 둔 그대로다.

문법 풀이

import 문법	뜻
(없음)	같은 파일의 함수는 import 없이 부른다

`_decode_token(token, "refresh")` 가 하는 검사 (05 와 같음):

순서	검사	실패하면
①	서명 · 만료 · 필수 칸(`exp` · `sub` · `type`), 알고리즘은 `HS256` 만	401
②	`type` 이 `"refresh"` 인지	401
③	`sub` 를 정수로	401

4. 흐름

1. 05 의 `_decode_token()` 에 종류 `"refresh"` 를 넘긴다.
2. 통과하면 회원 번호, 아니면 401.

5. 왜 이렇게

선택	이유
05 와 같은 검사 함수를 씀	서명 · 만료 · 알고리즘 고정(`alg: none` 차단) 규칙이 한 곳에만 있다. 05 를 고치면 이 함수도 같이 바뀐다
Access Token 을 거절	30분짜리 토큰으로 재발급을 받으면, 훔친 Access Token 으로 계속 새 토큰을 받아 30분 제한을 무력화할 수 있다
실패 이유와 상관없이 같은 401	05 와 같은 이유. 브라우저는 어떤 실패든 "다시 로그인" 하나로 처리한다
회원 상태(탈퇴 · 정지)는 여기서 보지 않음	이 함수는 토큰만 본다. 회원 확인은 33 라우터가 30 `get_active_user()` 로 한다 (06 `get_current_user` 와 같은 나눔)

6. 시험 (`tests/test_security.py` 에 8개 추가 · 모두 25개)

시험	확인한 것
만들고 읽기	`decode_refresh_token(create_refresh_token(7)) == 7` (27 과 짝)
Access Token 을 넣음	401 — 26 으로 만든 토큰은 거절
이상한 토큰 6개	만료 · 다른 비밀값 · `sub: "admin"` · `exp` 없음 · 엉터리 · 빈 글자 → 모두 401 "Please sign in again."

전체 시험 202개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 29 — `authenticate()` · `invalid_credentials()`

항목	내용
파일	`studionx/backend/app/services/auth_service.py`
작업	P1-05 로그인 — 이메일 · 비밀번호 확인 · PRD 01 FR-103 · FR-108
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	28 `decode_refresh_token()` · 30 `get_active_user()`

1. 하는 일

로그인 모달에 입력한 이메일 · 비밀번호가 맞는지 확인한다. 맞으면 회원을 돌려주고 **마지막 로그인 시각**을 기록한다.

경우	결과	로그인 모달 문구
이메일 · 비밀번호 맞음, 활동 회원	`User` (+ `last_login_at` = 지금)	— (로그인됨)
비밀번호 틀림	401 `INVALID_CREDENTIALS`	Email or password is incorrect.
없는 이메일	401 `INVALID_CREDENTIALS` — 위와 **같은 문구 · 같은 시간**	Email or password is incorrect.
탈퇴 회원 (비밀번호 맞아도)	401 `INVALID_CREDENTIALS` — 위와 같음	Email or password is incorrect.
정지 회원, 비밀번호 **맞음**	403 `ACCOUNT_DISABLED`	This account is disabled.
정지 회원, 비밀번호 틀림	401 `INVALID_CREDENTIALS`	Email or password is incorrect.

함수	역할
`authenticate()`	확인 · 기록
`invalid_credentials()`	401 오류를 만듦 (네 군데에서 같은 오류를 씀)

2. 언제 쓰나

쓰는 곳	화면
P1-05 `POST /auth/login` (32)	home.html 로그인 모달의 **Sign In** 버튼

32 가 이 함수로 회원을 확인한 뒤, 26 · 27 로 토큰 두 개를 만들어 준다.

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
```

```
from app.core.security import check_password_rules, hash_password, verify_password
from app.models import User
from app.models.user import UserStatus
from app.schemas.auth import PRIVACY_VERSION, AvailabilityResponse, SignupRequest
```

`auth_service.py` 맨 위의 import 이다. **`verify_password` 를 이번에 추가했다.** 이 함수는 `datetime` · `UTC` · `select` · `Session` · `AppError` · `ErrorCode` · `verify_password` · `User` · `UserStatus` 를 쓴다.

함수

```
# 없는 이메일로 로그인할 때 비교에 쓰는 가짜 해시 (bcrypt 비용 12 — 진짜 해시와 같은 시간이 걸림).
# 어떤 비밀번호와도 맞지 않는다. 비밀값이 아니므로 코드에 두어도 됨.
DUMMY_PASSWORD_HASH = "$2b$12$1YCmgxF3pcmPkm7MWpsgVuOD.w6LlO7o43SuOZLbLZkz.glHDfmva"


def authenticate(db: Session, email: str, password: str) -> User:
    user = db.scalar(select(User).where(User.email == normalize_email(email)))
    if user is None:
        verify_password(password, DUMMY_PASSWORD_HASH)  # 없는 이메일도 같은 시간(약 0.2초)이 걸리게
        raise invalid_credentials()
    if not verify_password(password, user.password_hash):
        raise invalid_credentials()
    if user.status == UserStatus.DELETED:
        raise invalid_credentials()
    if user.status == UserStatus.SUSPENDED:
        raise AppError(ErrorCode.ACCOUNT_DISABLED, 403, "This account is disabled.")
    user.last_login_at = datetime.now(UTC).replace(tzinfo=None)
    db.commit()
    return user


def invalid_credentials() -> AppError:
    return AppError(ErrorCode.INVALID_CREDENTIALS, 401, "Email or password is incorrect.")
```

문법 풀이

import 문법	뜻
`from app.core.security import …, verify_password`	16 의 비교 함수를 가져옴

식	뜻
`verify_password(password, DUMMY_PASSWORD_HASH)`	결과를 **쓰지 않고** 부르기만 함 — 시간만 쓰게 하려는 것
`raise invalid_credentials()`	함수가 만든 오류를 던짐 (21 의 `unavailable_error` 와 같은 방식)
`user.last_login_at = …` → `db.commit()`	회원 행의 마지막 로그인 시각을 바꿔 저장
`.replace(tzinfo=None)`	DB 칸(`DATETIME`)은 시간대 표시가 없어서 UTC 시각에서 표시를 뗌 (19 와 같음)

4. 흐름

1. 이메일을 17 로 맞춰 회원을 찾는다.
2. 없으면 → 가짜 해시로 한 번 비교(시간 맞추기) → 401.

3. 비밀번호가 틀리면 → 401.
4. 탈퇴 회원이면 → 401 (같은 문구).
5. 정지 회원이면 → 403.
6. 마지막 로그인 시각을 기록하고 회원을 돌려준다.

5. 왜 이렇게

선택	이유
틀린 이유를 알려 주지 않음	"이메일이 없음" 과 "비밀번호 틀림" 을 구분해 주면, 남의 이메일이 가입됐는지 알아낼 수 있다 (FR-103 통일 문구)
없는 이메일도 가짜 해시로 비교	비교(약 0.2초)를 건너뛰면 없는 이메일만 빨리 답한다. 응답 **시간**으로 가입 여부가 드러나는 것을 막는다 (16 §5 에 적어 둔 일)
가짜 해시를 코드에 상수로	서버가 켜질 때마다 0.2초씩 만들 필요가 없다. 비밀번호가 아니라 "아무것과도 맞지 않는 해시" 라 드러나도 문제없다. 비용 12 라 진짜 해시와 시간이 같다
탈퇴 회원도 같은 401	탈퇴한 계정이 있었다는 사실도 알려 주지 않는다 (18 의 추적 방지와 같은 생각)
정지(403)는 **비밀번호가 맞은 뒤에만** — 템플릿 순서 그대로	틀린 비밀번호로는 "이 이메일은 정지됐다" 를 알 수 없다. 본인(비밀번호를 아는 사람)에게만 정지 사실을 알려 준다
실패하면 `last_login_at` 을 바꾸지 않음	성공한 로그인만 기록한다

참고 — 탈퇴 3일 뒤: 18 로 비켜 둔 행은 이메일이 `deleted+…` 로 바뀌어 원래 이메일로는 찾을 수조차 없다. P1-09 정리 작업은 비밀번호 해시를 `"!"` 로 바꾸므로, 16 이 늘 `False` 를 준다.

6. 시험 (`tests/test_auth_service.py` 에 7개 추가 · 모두 44개, 실제 MySQL)

시험	확인한 것
성공	그 회원을 돌려주고 `last_login_at` 이 지금(5초 안)으로 기록됨
대소문자 · 공백	`" LOGIN@Example.com "` 로도 로그인됨 (17)
비밀번호 틀림	401 "Email or password is incorrect.", `last_login_at` 그대로
없는 이메일	같은 401 · 같은 문구, 걸린 시간 0.1초 이상 (가짜 해시 비교를 했음)
탈퇴 회원 · 맞는 비밀번호	401 (틀린 비밀번호와 구분 안 됨)
정지 회원	맞는 비밀번호 → 403 "This account is disabled.", 틀린 비밀번호 → 401
가짜 해시	`""` · `"password"` · `"Abcdef1!"` 등 어떤 것과도 맞지 않음

시험 도우미 `add_user()` 가 `password_hash` 를 바꿔 넣을 수 있게 고쳤다 (진짜 해시로 로그인 시험).

전체 시험 209개 통과, `ruff` 검사 통과. 시험 뒤 남은 행 0개.

Backend(FastAPI/Python) 30 — `get_active_user()`

항목	내용
파일	`studionx/backend/app/services/auth_service.py`
작업	P1-05 로그인 — 재발급 전 회원 확인 · PRD 01 FR-105 · FR-108
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	29 `authenticate()` · 31 리프레시 쿠키

1. 하는 일

회원 번호로 회원을 찾아, **아직 활동 중(`ACTIVE`)** 이면 돌려준다. 없거나, 정지 · 탈퇴했으면 401.

회원 번호의 회원	결과
활동 중	`User`
정지 (`SUSPENDED`) · 탈퇴 (`DELETED`)	401 `UNAUTHORIZED` "Please sign in again."
없음	401 (같음)

2. 언제 쓰나

쓰는 곳	하는 일
P1-05 `POST /auth/refresh` (33)	28 로 리프레시 토큰에서 회원 번호를 꺼낸 뒤, 이 함수로 회원을 확인하고 새 Access Token 을 줌

없으면 생기는 일

순서	일어나는 일
①	회원이 자동 로그인(3일 쿠키)으로 로그인
②	그날 탈퇴(P1-09) — 브라우저에는 리프레시 쿠키가 아직 남아 있음
③	다음 날 사이트를 열면 `api.js` 가 `/auth/refresh` 를 부름 → 토큰은 진짜이고 3일 안이라 28 은 통과
④	**이 함수가 없으면** 탈퇴한 회원에게 새 Access Token 이 나간다. 이 함수가 `DELETED` 를 보고 401 → 로그인 화면으로

정지(관리자가 `SUSPENDED` 로 바꿈)도 같다. 바꾼 순간부터 재발급이 막히고, 남은 Access Token(최대 30분)도 06 `get_current_user()` 가 막는다.

06 `get_current_user()` 와 비교

	06 `get_current_user()`	30 `get_active_user()`
회원 번호를 어디서	Access Token (05)	Refresh Token (28)
쓰는 곳	로그인이 필요한 모든 API	`/auth/refresh` 하나
검사	없거나 `ACTIVE` 아니면 401	같음

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.core.security import check_password_rules, hash_password, verify_password
from app.models import User
from app.models.user import UserStatus
from app.schemas.auth import PRIVACY_VERSION, AvailabilityResponse, SignupRequest
```

`auth_service.py` 맨 위의 import 이다 (이번에 추가한 것 없음). 이 함수는 **`Session`** · **`AppError`** · **`ErrorCode`** · **`User`** · **`UserStatus`** 를 쓴다.

함수

```
def get_active_user(db: Session, user_id: int) -> User:
    user = db.get(User, user_id)
    if user is None or user.status != UserStatus.ACTIVE:
        raise AppError(ErrorCode.UNAUTHORIZED, 401, "Please sign in again.")
    return user
```

문법 풀이

import 문법	뜻
`from app.models.user import UserStatus`	08 의 회원 상태 목록

식	뜻
`db.get(User, user_id)`	기본 키로 한 행. 없으면 `None` (06 과 같음)
`user is None or …`	앞이 참이면 뒤를 보지 않음 → `None.status` 오류가 나지 않음
`user.status != UserStatus.ACTIVE`	08 의 `StrEnum` 끼리 비교 (06 은 글자 `"ACTIVE"` 와 비교 — 결과는 같음)

4. 흐름

1. 회원 번호로 회원을 찾는다.
2. 없거나 활동 중이 아니면 401.
3. 회원을 돌려준다.

5. 왜 이렇게

선택	이유
`UNAUTHORIZED` 401 (403 아님)	`api.js` 는 재발급이 실패하면(401) 로그인 화면으로 보낸다. 정지 · 탈퇴 회원도 "다시 로그인" 으로 끝난다. 정지 사실은 로그인(29)을 다시 시도할 때 비밀번호가 맞으면 403 으로 알려 준다

선택	이유
없는 회원 · 정지 · 탈퇴를 같은 응답으로	05 · 06 과 같은 이유 — 이유를 알려 주지 않는다
06 은 고치지 않음	템플릿은 "06 이 이 함수를 불러 써도 됨" 이라고 했지만, 06 은 이미 검토가 끝난 코드이고 검사는 두 줄이다. 06 을 바꾸면 P1-03 문서 · 시험도 다시 봐야 한다

6. 시험 (`tests/test_auth_service.py` 에 4개 추가 · 모두 48개, 실제 MySQL)

시험	확인한 것
활동 회원	그 회원을 돌려줌
정지 · 탈퇴 (2개)	401 `UNAUTHORIZED` "Please sign in again."
없는 회원 번호	401

전체 시험 213개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 31 — `set_refresh_cookie()` · `clear_refresh_cookie()`

항목	내용
파일	`studionx/backend/app/api/v1/auth.py`
작업	P1-05 로그인 — 리프레시 쿠키 · PRD 01 FR-105 · FR-109
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	30 `get_active_user()` · 32 `POST /auth/login`

1. 하는 일

리프레시 토큰(27)을 브라우저 **쿠키**에 심고(`set`), 로그아웃할 때 지운다(`clear`). 응답 헤더 `Set-Cookie` 한 줄을 만든다.

호출	응답 헤더 `Set-Cookie`
`set_refresh_cookie(res, token, remember=True)`	`snx_refresh=eyJ…; HttpOnly; Max-Age=259200; Path=/api/v1/auth; SameSite=lax`
`set_refresh_cookie(res, token, remember=False)`	`snx_refresh=eyJ…; HttpOnly; Path=/api/v1/auth; SameSite=lax`
`clear_refresh_cookie(res)`	`snx_refresh=""; expires=(지금); HttpOnly; Max-Age=0; Path=/api/v1/auth; SameSite=lax`

쿠키 속성	값	뜻
`HttpOnly`	켬	**JS 가 읽을 수 없음** (`document.cookie` 에 안 보임). 스크립트 공격(XSS)이 훔쳐 가지 못함
`Path=/api/v1/auth`		`/api/v1/auth/…` 요청에만 실림. 다른 API · 페이지 요청에는 안 실림
`SameSite=lax`		다른 사이트가 몰래 보낸 `POST` 에는 안 실림 (CSRF 대비)
`Max-Age`	259200초(3일) 또는 없음	없으면 **세션 쿠키** — 브라우저를 닫으면 사라짐
`Secure`	배포(`APP_ENV=prod`)에서만	HTTPS 로만 보냄. 개발(`http://localhost`)에서 켜면 쿠키가 아예 안 실려서 끔

2. 언제 쓰나

쓰는 곳	함수	화면
32 `POST /auth/login`	`set_refresh_cookie(…, remember=data.remember)`	로그인 모달 — **Keep me signed in** 체크 여부
34 `POST /auth/logout`	`clear_refresh_cookie()`	헤더 **Sign Out** (JS 08)

Keep me signed in	브라우저를 닫았다 다시 열면
체크	3일 동안 쿠키가 남아 있음 → `api.js` 가 재발급 → 로그인된 채로

Keep me signed in	브라우저를 닫았다 다시 열면
체크 안 함	쿠키가 사라짐 → 재발급 실패 → 로그인 화면

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import APIRouter, Cookie, Query, Response, status
from pydantic import EmailStr

from app.api.deps import Db
from app.core.config import get_settings
from app.schemas.auth import (
    AvailabilityResponse,
    LoginRequest,
    Nickname,
    SignupRequest,
    SignupResponse,
    TokenResponse,
)
from app.schemas.common import ErrorResponse
from app.services import auth_service
```

`auth.py` 맨 위의 import 이다. **`from app.core.config import get_settings` 를 이번에 추가했다.** 이 함수들은 `Response` · `get_settings` 를 쓴다.

함수

```
REFRESH_COOKIE = "snx_refresh"
REFRESH_COOKIE_PATH = "/api/v1/auth"


def set_refresh_cookie(response: Response, token: str, remember: bool) -> None:
    settings = get_settings()
    if remember:
        max_age = settings.jwt_refresh_days * 24 * 60 * 60
    else:
        max_age = None
    response.set_cookie(
        key=REFRESH_COOKIE,
        value=token,
        path=REFRESH_COOKIE_PATH,
        httponly=True,
        samesite="lax",
        secure=settings.app_env == "prod",
        max_age=max_age,
    )


def clear_refresh_cookie(response: Response) -> None:
    response.delete_cookie(
        REFRESH_COOKIE,
```

```
        path=REFRESH_COOKIE_PATH,
        httponly=True,
        samesite="lax",
        secure=get_settings().app_env == "prod",
    )
```

문법 풀이

import 문법	뜻
`from app.core.config import get_settings`	`.env` 를 읽은 설정 (05 와 같음)

식	뜻
`response: Response`	라우터가 받은 응답 객체. 여기에 쿠키를 붙이면 응답 헤더로 나감
`response.set_cookie(key=…, value=…, …)`	`Set-Cookie` 헤더 한 줄을 만듦
`max_age=None`	`Max-Age` 를 아예 쓰지 않음 → 세션 쿠키
`settings.app_env == "prod"`	배포 환경이면 `True`
`response.delete_cookie(…)`	빈 값 + `Max-Age=0` 인 같은 이름의 쿠키를 보내 브라우저가 지우게 함
`-> None`	돌려주는 값 없음 — `response` 를 바꾸는 것이 일

4. 흐름

심기: 수명을 정한다(체크면 3일, 아니면 없음) → 속성을 붙여 `Set-Cookie` 를 만든다. **지우기:** 같은 이름 · 같은 경로로 "지금 만료" 쿠키를 보낸다.

5. 왜 이렇게

선택	이유
쿠키에 담음 (응답 본문 아님)	본문으로 주면 JS 가 어딘가에 저장해야 한다. `HttpOnly` 쿠키는 JS 가 손댈 수 없고, 브라우저가 알아서 보낸다
경로를 `/api/v1/auth` 로 좁힘	3일짜리 토큰이 모든 요청에 실려 다니지 않는다. 재발급 · 로그아웃할 때만 서버에 간다
체크 안 하면 세션 쿠키	공용 컴퓨터에서 브라우저를 닫으면 로그인이 풀린다 (FR-109)
`Secure` 는 배포에서만	개발 서버는 `http://` 라 `Secure` 쿠키가 실리지 않는다. 배포(HTTPS)에서는 반드시 켠다
지울 때도 같은 속성	브라우저는 이름 · 경로가 같아야 같은 쿠키로 보고 지운다. 템플릿은 경로만 맞췄지만, 속성까지 같게 해 두면 브라우저마다 다르게 처리할 걱정이 없다

6. 시험 (`tests/test_auth_cookies.py`, 새로 만듦 · 5개)

시험	확인한 것
체크	`snx_refresh=TOKEN`, `Max-Age=259200`, `Path=/api/v1/auth`, `HttpOnly`, `SameSite=lax`
체크 안 함	`Max-Age` · `expires` 없음 (세션 쿠키), `HttpOnly`

시험	확인한 것
`Secure` (2개)	설정을 `dev` 로 바꾸면 없음, `prod` 로 바꾸면 있음
지우기	빈 값, `Max-Age=0`, 같은 경로

전체 시험 218개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 32 — 라우터 `POST /api/v1/auth/login`

항목	내용
파일	`studionx/backend/app/api/v1/auth.py` (함수 이름 `login`)
작업	P1-05 로그인 — 로그인 API · PRD 01 FR-103 · FR-105 · FR-109
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	31 리프레시 쿠키 · 33 `POST /auth/refresh`

1. 하는 일

이메일 · 비밀번호를 확인하고, **토큰 두 개**를 나눠서 준다.

```
POST /api/v1/auth/login
{"email": "route.user@example.com", "password": "Abcdef1!", "remember": true}

→ 200
Set-Cookie: snx_refresh=eyJ…(3일); HttpOnly; Max-Age=259200; Path=/api/v1/auth; SameSite=lax
{"access_token": "eyJ…(30분)", "token_type": "bearer", "expires_in": 1800}
```

토큰	어디로	만든 함수
Access (30분)	응답 **본문** → JS 메모리	26
Refresh (3일)	응답 **쿠키** (`HttpOnly`) → JS 는 못 봄	27 · 31

경우	응답
맞음	200 + 위와 같음
비밀번호 틀림 · 없는 이메일 · 탈퇴	401 `INVALID_CREDENTIALS` "Email or password is incorrect." — **쿠키 없음**
정지 회원 (비밀번호 맞음)	403 `ACCOUNT_DISABLED`
칸 빠짐	422

2. 언제 쓰나

home.html 로그인 모달의 **Sign In** 버튼 → `home.js` / `auth-ui.js` (P1-07).

순서	화면 · 서버	일어나는 일
①	로그인 모달	이메일 · 비밀번호 · **Keep me signed in**
②	P1-07 JS	`api.post('/auth/login', {email, password, remember}, {auth: false})`
③	**이 라우터**	29 확인 → 27 · 31 쿠키 → 26 본문
④	P1-07 JS	`setAccessToken(access_token)` (JS 01) → 헤더가 My Page · My Projects 로 바뀜 → `next` 페이지로 이동 (JS 05)

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import APIRouter, Cookie, Query, Response, status
from pydantic import EmailStr

from app.api.deps import Db
from app.core.config import get_settings
from app.core.security import create_access_token, create_refresh_token
from app.schemas.auth import (
    AvailabilityResponse,
    LoginRequest,
    Nickname,
    SignupRequest,
    SignupResponse,
    TokenResponse,
)
from app.schemas.common import ErrorResponse
from app.services import auth_service
```

`auth.py` 맨 위의 import 이다. **`from app.core.security import create_access_token, create_refresh_token` 을 이번에 추가했다.** 이 함수는 `Response` · `Db` · `get_settings` · 두 토큰 함수 · `LoginRequest` · `TokenResponse` · `auth_service` 를 쓴다.

함수

```
@router.post(
    "/login",
    response_model=TokenResponse,
    responses={401: {"model": ErrorResponse}, 403: {"model": ErrorResponse}},
)
def login(data: LoginRequest, response: Response, db: Db) -> TokenResponse:
    user = auth_service.authenticate(db, data.email, data.password)
    set_refresh_cookie(response, create_refresh_token(user.user_id), data.remember)
    return TokenResponse(
        access_token=create_access_token(user.user_id),
        expires_in=get_settings().jwt_access_minutes * 60,
    )
```

`@router.post(…)` 와 함수 머리는 템플릿에 있었고, 몸통 세 줄을 채웠다.

문법 풀이

import 문법	뜻
`from app.core.security import A, B`	26 · 27 의 토큰 함수 두 개를 가져옴

식	뜻
`response: Response`	FastAPI 가 넣어 주는 응답 객체. 여기에 쿠키를 붙이면(31) 최종 응답 헤더에 실림

식	뜻
`data: LoginRequest`	본문 `{email, password, remember}` (`remember` 는 없으면 `False`)
`TokenResponse(access_token=…, expires_in=1800)`	응답 본문. `token_type` 은 기본값 `"bearer"`

4. 흐름

1. 29 `authenticate()` — 틀리면 여기서 401 · 403 로 끝남 (쿠키도 토큰도 만들지 않음).
2. 27 로 리프레시 토큰 → 31 로 쿠키에 담음 (`remember` 에 따라 3일 · 세션).
3. 26 으로 Access Token → 본문으로 200.

5. 왜 이렇게

선택	이유
확인을 맨 먼저	실패하면 토큰 · 쿠키를 만들지 않는다. 시험에서 실패 응답에 `Set-Cookie` 가 없음을 확인했다
Access 는 본문, Refresh 는 쿠키	27 §5 — 짧은 토큰은 JS 가 쓰고, 긴 토큰은 JS 가 못 보게 숨긴다
`expires_in` 을 줌	화면이 "몇 초 뒤 만료" 를 알 수 있다. 지금 `api.js` 는 401 을 받으면 재발급하므로 쓰지 않지만, 미리 재발급하는 방식으로 바꿀 때 쓸 수 있다
이메일 형식 검사 (`EmailStr`)를 하지 않음	로그인 칸에 형식이 틀린 이메일을 넣어도 422 가 아니라 401 "Email or password is incorrect." 하나로 답한다. 무엇이 틀렸는지 알려 주지 않는 원칙(29)과 같다

6. 시험 (`tests/test_auth_routes.py` 에 7개 추가 · 모두 34개, 실제 앱 + MySQL)

가입(22)으로 회원을 만든 뒤 로그인했다.

시험	확인한 것
200 본문	칸이 `access_token` · `token_type` · `expires_in` 셋뿐 (리프레시 토큰 없음), `bearer` · 1800, 05 로 읽으면 그 회원 번호
쿠키 (체크 · 안 체크)	`snx_refresh=…; HttpOnly; Path=/api/v1/auth`, 체크면 `Max-Age=259200`, 안 체크면 없음. 쿠키 값을 28 로 읽으면 그 회원 번호
마지막 로그인	`last_login_at` 이 기록됨
비밀번호 틀림 · 없는 이메일	둘 다 401 · 같은 본문, `Set-Cookie` 없음
비밀번호 칸 빠짐	422

전체 시험 225개 통과, `ruff` 검사 통과. 시험 뒤 남은 행 0개.

Backend(FastAPI/Python) 33 — 라우터 `POST /api/v1/auth/refresh`

항목	내용
파일	`studionx/backend/app/api/v1/auth.py` (함수 이름 `refresh`)
작업	P1-05 로그인 — Access Token 재발급 · PRD 01 FR-105 · FR-108
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	32 `POST /auth/login` · 34 `POST /auth/logout`

1. 하는 일

브라우저에 남은 **리프레시 쿠키**로 새 Access Token(30분)을 준다. 요청 본문은 없다 — 쿠키가 자동으로 실려 온다.

```
POST /api/v1/auth/refresh
Cookie: snx_refresh=eyJ…

→ 200 {"access_token": "eyJ…(새 30분)", "token_type": "bearer", "expires_in": 1800}
```

쿠키	응답	화면 (`api.js`)
정상 (3일 안, 활동 회원)	200 + 새 Access Token	메모리에 저장, 하던 요청 계속
없음 (로그인 안 한 방문자)	401 "Please sign in."	로그인 안 된 것으로 처리 (정상 흐름)
만료 · 위조 · Access Token · 엉터리	401 "Please sign in again."	로그인 화면으로
진짜지만 회원이 탈퇴 · 정지	401 "Please sign in again."	로그인 화면으로

2. 언제 쓰나

P1-02 의 `refreshAccessToken()` 이 부른다. **이 라우터가 생겨서 P1-02 의 재발급이 실제로 동작한다.**

상황	누가 부르나
페이지를 새로 열었을 때 (메모리 토큰이 비어 있음)	`apiFetch` 가 요청 전에 먼저 (JS 02)
30분이 지나 API 가 401 을 줄 때	`apiFetch` 가 재발급 후 같은 요청을 한 번 다시
헤더 버튼을 그릴 때 · 로그인 전용 페이지를 열 때	`isLoggedIn()` · `requireLogin()` (JS 06 · 07)

ai-note 34(P1-02 · 03 실제 사이트 시험)에서 `/auth/refresh → 500` 이었던 것이 이제 200 · 401 로 바뀐다.

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import APIRouter, Cookie, Query, Response, status
from pydantic import EmailStr
```

```
from app.api.deps import Db
from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
from app.core.security import create_access_token, create_refresh_token, decode_refresh_token
from app.schemas.auth import (…, TokenResponse)
from app.schemas.common import ErrorResponse
from app.services import auth_service
```

`auth.py` 맨 위의 import 이다. **`AppError` · `ErrorCode` · `decode_refresh_token` 을 이번에 추가했다.** 이 함수는 `Annotated` · `Cookie` · `Response` · `Db` · `get_settings` · `AppError` · `ErrorCode` · `create_access_token` · `decode_refresh_token` · `TokenResponse` · `auth_service` 를 쓴다.

함수

```
@router.post(
    "/refresh",
    response_model=TokenResponse,
    responses={401: {"model": ErrorResponse}},
)
def refresh(
    response: Response,
    db: Db,
    refresh_token: Annotated[str | None, Cookie(alias=REFRESH_COOKIE)] = None,
) -> TokenResponse:
    if refresh_token is None:
        raise AppError(ErrorCode.UNAUTHORIZED, 401, "Please sign in.")
    user_id = decode_refresh_token(refresh_token)
    user = auth_service.get_active_user(db, user_id)
    return TokenResponse(
        access_token=create_access_token(user.user_id),
        expires_in=get_settings().jwt_access_minutes * 60,
    )
```

함수 머리는 템플릿에 있었고, 몸통을 채웠다.

문법 풀이

import 문법	뜻
`from fastapi import …, Cookie, …`	요청의 쿠키에서 값을 꺼내는 표시 (템플릿에 이미 있었음)

식	뜻
`Annotated[str | None, Cookie(alias="snx_refresh")]`	쿠키 `snx_refresh` 의 값을 `refresh_token` 인자로 받음. 파이썬 이름과 쿠키 이름이 달라서 `alias` 로 연결
`= None`	쿠키가 없으면 `None` (오류 아님)
`response: Response`	지금은 쓰지 않음 — 나중에 리프레시 토큰을 갈아 끼울 때(아래 §5) 쿠키를 심는 데 씀

4. 흐름

1. 쿠키가 없으면 401 "Please sign in."
2. 28 로 토큰 검사 → 회원 번호 (실패하면 401 "Please sign in again.")

3. 30 으로 회원이 아직 활동 중인지 (아니면 401)
4. 26 으로 새 Access Token → 200

5. 왜 이렇게

선택	이유
쿠키 없음과 토큰 이상을 다른 문구로	쿠키가 없는 것은 "처음부터 로그인 안 함" — 정상이다. 쿠키가 있는데 안 되는 것은 "로그인이 풀림". 코드는 둘 다 `UNAUTHORIZED` 라 `api.js` 처리는 같다 (06 과 같은 나눔)
토큰 검사(28) 다음에 회원 확인(30)	토큰은 진짜여도 그 사이 탈퇴 · 정지했을 수 있다 (30 §2)
리프레시 토큰을 갈아 끼우지 않음 (템플릿의 선택 사항)	갈아 끼우면 재발급할 때마다 3일이 다시 늘어나 사실상 영원히 로그인된다. 또 체크 여부(`remember`)를 토큰에 따로 담아야 한다. 지금은 **로그인 후 3일이면 다시 로그인**으로 단순하게 둔다
실패해도 쿠키를 지우지 않음	오류 응답은 01 처리기가 새로 만들어서, 여기서 붙인 쿠키가 실리지 않는다. 쿠키는 3일이 지나면 브라우저가 지우고, 로그아웃(34)이 지운다

6. 시험 (`tests/test_auth_routes.py` 에 6개 추가 · 모두 40개, 실제 앱 + MySQL)

시험 도구(`TestClient`)는 브라우저처럼 로그인 응답의 쿠키를 기억했다가 다음 요청에 실어 보낸다.

시험	확인한 것
로그인 후 재발급 (체크 · 안 체크)	200, `bearer` · 1800, 새 토큰을 05 로 읽으면 그 회원 번호
쿠키 없음	401 "Please sign in."
엉터리 쿠키 · Access Token 을 쿠키에	401 `UNAUTHORIZED` "Please sign in again."
로그인 뒤 탈퇴	쿠키는 남아 있어도 401 (30 이 막음)

전체 시험 231개 통과, `ruff` 검사 통과. 시험 뒤 남은 행 0개.

Backend(FastAPI/Python) 34 — 라우터 `POST /api/v1/auth/logout`

항목	내용
파일	`studionx/backend/app/api/v1/auth.py` (함수 이름 `logout`)
작업	P1-05 로그인 — 로그아웃 · PRD 01 FR-104 (P1-05 마지막)
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	33 `POST /auth/refresh` · (P1-05 끝) · 실제 사이트 시험 ai-note 37

1. 하는 일

리프레시 쿠키를 **지우라는 응답**을 보낸다. 본문 없이 204.

```
POST /api/v1/auth/logout

→ 204 No Content
Set-Cookie: snx_refresh=""; expires=(지금); HttpOnly; Max-Age=0; Path=/api/v1/auth; SameSite=lax
```

토큰	로그아웃할 때 누가 지우나
Refresh (쿠키)	**이 라우터** — JS 는 `HttpOnly` 쿠키를 지울 수 없어서 서버가 지워야 함
Access (JS 메모리)	화면 `logout()` 의 `clearAccessToken()` (JS 08)

2. 언제 쓰나

헤더의 **Sign Out** 버튼 → JS 08 `logout()`.

순서	화면 · 서버	일어나는 일
①	`logout()`	`api.post('/auth/logout', undefined, { auth: false })`
②	**이 라우터**	204 + 쿠키 지우기
③	`logout()`	`clearAccessToken()` → `home.html` 로 이동
④	다음에 사이트를 열면	쿠키가 없어 재발급(33) 401 → 로그인 안 된 상태

ai-note 34(P1-02 · 03 실제 사이트 시험) 에서 `/auth/logout → 500` 이었던 것이 이제 204 다. (그때도 `logout()` 은 실패를 무시하고 홈으로 갔지만, 쿠키는 남아 있었다.)

3. 코드

import (파일 맨 위 — 32 · 33 과 같음, 이번에 추가한 것 없음)

```
from fastapi import APIRouter, Cookie, Query, Response, status
…
```

이 함수는 **`Response`** · **`status`** 를 쓰고, 쿠키 지우기는 31 `clear_refresh_cookie()` 에 맡긴다.

함수

```
@router.post("/logout", status_code=status.HTTP_204_NO_CONTENT)
def logout(response: Response) -> None:
    clear_refresh_cookie(response)
```

문법 풀이

import 문법	뜻
`from fastapi import status`	상태 번호에 이름을 붙인 모음 (`HTTP_204_NO_CONTENT` = 204)

식	뜻
`status_code=204`	성공 · **본문 없음**
`-> None`	돌려줄 본문이 없음
`response: Response`	여기에 붙인 쿠키 지우기 헤더가 204 응답에 실림
`db` · `user` 인자가 없음	로그인 확인도, DB 도 쓰지 않음

4. 흐름

1. 31 로 "지금 만료" 쿠키를 응답에 붙인다.
2. 204 로 응답한다.

5. 왜 이렇게

선택	이유
로그인 확인 없이 받음 (템플릿 권장)	Access Token 이 이미 만료된 채로 Sign Out 을 눌러도 쿠키는 지워져야 한다. 하는 일이 "쿠키 지우기" 뿐이라 누가 불러도 위험이 없다
서버에 아무것도 기록하지 않음	토큰을 서버에 저장하지 않는 방식(JWT)이다. 남은 Access Token 은 최대 30분 뒤 스스로 만료된다
204	돌려줄 내용이 없다는 표준 응답. `api.js` 는 204 를 `null` 로 처리한다 (JS 02)

참고 — 쿠키를 훔쳐 간 경우: 로그아웃은 **이 브라우저**의 쿠키만 지운다. 이미 복사해 간 리프레시 토큰은 3일 동안 유효하다. 이를 막으려면 서버에 "무효 토큰 목록" 을 두거나 비밀번호 변경 때 모든 토큰을 끊는 기능이 필요하다 — Phase 1 범위 밖이라 나중에 검토한다.

6. 시험 (`tests/test_auth_routes.py` 에 3개 추가 · 모두 43개, 실제 앱 + MySQL)

시험	확인한 것
로그인 → 로그아웃 → 재발급	로그아웃 전 재발급 200, 로그아웃 204 · 본문 없음 · `snx_refresh=""; Max-Age=0; Path=/api/v1/auth`, 로그아웃 뒤 재발급 401
로그인 안 한 상태	204
엉터리 `Authorization` 헤더	204 (로그인 확인을 하지 않음)

전체 시험 234개 통과, `ruff` 검사 통과. 인증 파일(`auth.py` · `security.py` · `auth_service.py`)에 남은 `NotImplementedError` 0개.